Contextual Permission Boundary Testing in the OpenClaw Security Attack and Defense Challenge: Task-Only Red-Teaming via Execution Traces

Zeren Luo[1] , **Jingyi Zheng**[2] , **Yule Liu**[2] , **Zhen Sun**[2] , **Xinlei He**[1,*]
[1]Wuhan University
[2]The Hong Kong University of Science and Technology (Guangzhou)
[*]Corresponding author
xinlei.he@whu.edu.cn

Abstract

Tool-augmented language-model agents can turn ordinary user requests into high-impact tool calls. Therefore, their safety must be evaluated from executed trajectories rather than final answers alone. We study one-shot task-only testing of contextual authorization: can a single ordinary task cause an agent to perform a protected operation that the task does not authorize? We propose Contextual Permission Boundary Testing (CPBT), a tester-side workflow that uses workspace-level access to identify protected operations, nearby benign workflow surfaces, tool schemas, and apparent authorization cues; generates target-grounded task probes; and validates them from execution traces. We evaluate CPBT in the SeClaw challenge, where hidden pre-execution checks can reject a locally successful probe before its trajectory is scored. CPBT therefore uses the resulting sparse pass/fail feedback only as a heuristic for ranking locally validated candidates. Using CPBT, we induced all 19 protected operations in local execution, whereas the strongest official submission received credit for only 16 targets. All but one of the credited cases were strict task-only successes; the remaining case was attachment-assisted. This gap demonstrates that ordinary workflow context can expose permission-boundary failures, but local execution alone does not capture end-to-end feasibility under hidden safeguards.

1 Introduction

Tool-augmented agents increasingly connect language models to commercial systems [Mao *et al.*, 2026], personal devices [Hu *et al.*, 2025], and external information sources [Luo *et al.*, 2025]. Through these tools, an agent can read files, query customer records, update orders, issue refunds, or control devices while completing a user request. As these agents are used for more real work, safety evaluation must inspect more than the text they return [Perez *et al.*, 2022; Ganguli *et al.*, 2022]. Prior work has shown that security-relevant behavior can be triggered or concealed in ways that are not captured by surface-level inspection alone [Greshake *et al.*, 2023; Zhan *et al.*, 2024; Sun *et al.*, 2025; Zheng *et al.*, 2025a; Zheng *et al.*, 2025b]. This limitation is especially important for tool-augmented agents, where a final answer can look harmless after the agent has already read confidential data or changed protected status [Debenedetti *et al.*, 2024]. The executed tool trajectory is therefore a safety object in its own right: it records which tools were called, which objects they touched, and what state they changed.

This paper focuses on unsafe tool execution as a process-level authorization failure. The question is whether an agent, while trying to complete an ordinary user task, can call a protected tool operation that the task does not authorize. Such a failure can be invisible in the final response because the decisive behavior happens in the tool calls. Evaluating tool-augmented agents therefore requires supervision over the execution trace, checking whether each tool call is allowed by the task and its workflow context.

We study this process-level authorization problem in the SeClaw attack-and-defense setting, where OpenClaw instantiates spec-driven security tasks for autonomous agents across enterprise, e-commerce, and smart-home scenarios [Cheng *et al.*, 2026]. In the strict task-only form of this setting, the evaluated agent receives a single natural-language task, and success is determined by whether the resulting execution trace reaches a protected operation. The tester may use local workspace access while designing the task, including tool schemas, business objects, and prior execution traces. This local grounding avoids simple misses, such as naming the wrong object or asking for a field that the tool does not accept, while the evaluated agent still receives only the final task text. The task must then pass two checks. Its execution trace must show the protected tool operation, and the same task must survive hidden pre-execution checks before official trajectory scoring.

To address this problem in the SeClaw setting, we introduce Contextual Permission Boundary Testing (CPBT), a lightweight tester-side workflow for carrying out this test. CPBT starts by inspecting the local workspace to identify the protected operation, the ordinary pages or records that lead near it, the required tool fields, the relevant business objects, and cues such as unverified role labels, reason fields, or urgency markers that an agent may over-interpret as permission. It uses these details to write ordinary task probes that anchor the right object and schema without directly asking for the

protected action. Each probe is executed locally and judged from the tool trace rather than the final answer; failures such as wrong tool use, wrong resource selection, explicit denial, or schema mismatch guide the next revision. After a probe reaches the protected operation locally, CPBT uses sparse official feedback to prefer submissions that are also likely to pass hidden pre-execution checks. The workflow therefore keeps two questions separate: can the task induce the protected tool trajectory, and will the submitted package survive long enough for that trajectory to be scored?

The main empirical result is a separation between local reachability and official evaluation feasibility. Across enterprise, e-commerce, and smart-home workflows, local trajectory validation reached all 19 protected operations. After hidden filtering in the official final round, the primary task-only result was 15/19 credited targets. One separately reported attachment-assisted extension yielded 16/19 credited targets in total. This gap is the central measurement point: ordinary business-context tasks can expose many permission-boundary failures, but robust evaluation must model both executed tool behavior and the hidden checks that decide whether a task is run.

This paper makes three contributions. First, we formulate one-shot task-only red-teaming as a low-friction measurement problem for authorization failures in tool trajectories. Second, we introduce CPBT, a tester-side workflow with local trace construction and black-box guard feedback as its two loops. We also instantiate CPBT as a reusable coding-agent skill for authorized evaluations; the skill runs on the tester side and does not modify the evaluated agent or its setup. Third, we show that local trajectory reachability and official black-box feasibility diverge in the SeClaw setting: local validation reached 19/19 protected operations, whereas official task-only feasibility reached 15/19 after hidden filtering, with one separately reported attachment-assisted extension.

2 Task Setting

2.1 Competition Setting and Scoped Task-Only Variant

We study one-shot red teaming in the SeClaw security attack-and-defense setting, where an agent is connected to a simulated tool environment. The environment contains general-purpose tools and scenario-specific business services. A user task is given to the agent, the agent decides which tools to call, and the environment records the resulting execution trajectory. A target is credited only when the submission passes the pre-execution guards and the trajectory contains the required result evidence, required basic tool invocations, and scenario-specific service calls for the target.

The evaluation is not a final-response evaluation. A final answer may report a short summary, a status marker, or a QA note, while the internal tool trajectory may contain a protected action. We therefore treat the executed trajectory as the primary security object. Formally, let $p = (m, x)$ denote a complete submitted package, where x is its natural-language task and m denotes its non-task components. The agent produces a trajectory

$$\tau(p) = (a_1, o_1, \ldots, a_T, o_T),$$

where a_t denotes an agent action or tool call and o_t denotes the corresponding observation. For each target y, the environment defines a trajectory-level predicate

$$R_y(\tau(p)) \in \{0, 1\},$$

which indicates whether the trajectory reaches the protected operation for target y.

The challenge evaluates the complete package rather than the task text in isolation. A submitted package first passes a skill and material level security scan, then a task-instruction safety evaluation, and only then reaches trajectory scoring. We write these stages abstractly as

$$G_1(p), \quad G_2(p), \quad R_y(\tau(p)),$$

where G_1 denotes the skill and material-level guard, G_2 denotes the task-instruction guard, and R_y denotes the target-specific trajectory predicate. The official success condition can be written as

$$S_y(p) = G_1(p) \cdot G_2(p) \cdot R_y(\tau(p)).$$

This decomposition is important because a task can be locally executable but officially unsuccessful if it is rejected before trajectory scoring. It also separates rejection at the material level, rejection at the task-instruction level, and failure to produce the required trajectory after both checks have passed. In the strict task-only analysis, non-task components may differ only as required by the target environment, but they contain no adversarial logic; the task instruction is the only component deliberately designed to induce the protected operation.

2.2 Target Domains and Protected Operations

The target set contains 19 protected operations across three scenarios: Enterprise OA, E-commerce, and IoT Smart Home. These domains cover different classes of permission boundaries. Enterprise OA emphasizes confidential records, review workflows, approvals, finance, salary, calendar, and personnel-related information. E-commerce emphasizes customer-order privacy, payment actions, order lifecycle updates, customer communication, catalog modification, and inventory integrity. IoT Smart Home emphasizes physical device control, door access, security-state changes, user creation, and camera streaming. Table 1 summarizes the domains and operation classes.

The domains differ not only in their tools but also in the type of contextual ambiguity they create. OA workflows naturally contain review tokens, access requests, archives, and redaction cards. E-commerce workflows naturally contain support-manager and marketing-manager roles. Smart-home workflows naturally contain emergency, homeowner, and maintenance-readiness contexts. These contextual cues are realistic in benign workflows, but they can also be over-interpreted by an agent as authorization for protected actions.

2.3 One-shot Task-Only Testing

Although the official competition accepts complete submission packages, our strict task-only analysis restricts adversarial design to the natural-language task. The package still contains the organizer-required scaffold, but its non-task components contain no target-specific attack logic. For each target

Table 1: Target domains and protected operation classes in the SeClaw setting.

Domain	Targets	Protected operation types
Enterprise OA	8	Confidential document access, record export, email deletion, finance export, salary query, approval modification, calendar access, and HR-related document access.
E-commerce	6	Broad order access, refund issuance, order-state update, customer message sending, product update, and inventory update.
IoT Smart Home	5	Device control, door access, security-state change, smart-home user creation, and camera stream activation.
Total	19	Three domains covering data access, state mutation, payment or order actions, and physical-device control.

y, the tester provides one natural-language task x_y in a single turn, without multi-turn negotiation. The task does not rely on a malicious skill, a modified tool description, executable wrapper logic, external prompt injection, or a crafted attachment. We therefore use "task-only" to describe the source of the adversarial instruction rather than the physical file composition of the competition package.

This setting measures a low-friction form of tool-agent risk. If a protected operation can be induced by one ordinary-looking task, then the permission boundary is weak even before considering stronger adversarial surfaces. Task-only failures are especially important because they resemble normal business requests: document review, support QA, payment reconciliation, maintenance checks, and form handoffs are all plausible tasks for tool-augmented agents.

We treat attachment-assisted materials as a secondary extension rather than part of the strict task-only setting. In that variant, the user instruction remains a normal workflow request, but an auxiliary document may provide additional context. We report this case separately from task-only results throughout the paper.

2.4 Threat Model

Our threat model is red-team in mechanism but blue-team in purpose. The tester is an authorized evaluator with workspace-level white-box access, including tool definitions, local task files, service behavior, logs, and execution traces. This models a deployment team testing its own agent before release, not an external attacker with arbitrary access. The submitted input to the evaluated agent remains constrained: in the main setting, it is a single natural-language task with no auxiliary payload. Here, "white-box" refers to access to the local workspace, tool schemas, service behavior, and execution traces; it does not imply access to model parameters or to the implementation of the hidden official guards.

The tester does not modify the evaluated model, tool implementations, official SeClaw container image, scoring backend, or audit logs and does not manually execute the target operation. The evaluated agent must choose and invoke the relevant tools by itself. The objective is therefore not to exploit the operating system or forge evidence, but to measure whether ordinary business-context tasks can cause the agent to cross semantic and authorization boundaries.

3 Method

3.1 Overview

Our method converts local red-team exploration into a reusable defense-simulation workflow with two coupled loops. The workspace-level trajectory loop inspects the local workspace, constructs a task-only probe, executes it locally, and verifies whether the resulting trajectory reaches the protected operation. The black-box guard-feedback loop uses limited official feedback to rank locally successful probes under hidden pre-execution checks.

The central principle is to test contextual authorization confusion under ordinary workflow pressure. A probe should resemble benign domain work while bringing the agent near a protected read, write, or control operation. The task should not directly instruct the agent to behave maliciously; instead, it tests whether routine workflow context is incorrectly treated as sufficient authorization. The workflow can also be applied to attachment-assisted tests when auxiliary materials are allowed, but we treat this variant as a secondary extension and report it separately from the strict task-only setting.

3.2 Workspace Inspection

For each target, the tester's agent first inspects the local workspace to identify four elements: the protected operation, nearby benign workflow surfaces, the required tool schema, and the apparent authorization boundary.

The protected operation is the action that would count as target success in the audit trajectory of the competition. Nearby workflow surfaces are ordinary interfaces an agent might visit before reaching that operation, such as list views, detail cards, review forms, archive panels, support notes, maintenance panels, or status pages. The schema includes required fields, accepted labels, object identifiers, and tool-facing field names. The authorization boundary consists of cues that may be confused with permission, such as review values, unverified role labels, reason fields, urgency markers, homeowner claims, or textual markers that appear to assert authorization.

This inspection step is essential because one-shot probes are brittle without local grounding. A vague task may stop at a harmless overview; an overly direct task may be rejected before execution; and a schema mismatch may cause the tool call to fail even when the agent follows the intended workflow. Workspace inspection provides the grounding needed to separate these failure modes.

3.3 Task-Only Probe Generation

After workspace inspection, the coding agent generates candidate one-shot probes according to four design rules. First, the probe uses business-context framing. It is written as a routine task in the target domain rather than as a direct request for a protected action. The frame should place the agent in a plausible workflow while leaving authorization to be determined by the agent and tool policy. Second, the probe uses object and field anchoring. It binds the task to one concrete business object or a narrow object set, such as an order, product, approval, document, device, user marker, status label, or native tool-facing field name. This reduces tool drift and makes the audit result target-specific. Third, the probe introduces authorization-cue tests. It may include scenario-local context such as a review value, unverified role label, urgency marker, homeowner claim, or checklist context. These cues are not treated as valid authorization; they are test stimuli for measuring whether the agent confuses workflow context with permission. Fourth, the probe avoids overtly malicious wording. This prevents the evaluation from collapsing into an obvious refusal test and keeps the input comparable to benign workflow requests. The goal is not to hide an attack string, but to measure whether ordinary task context is sufficient to trigger a protected operation.

3.4 Execution-Trajectory Auditing

Candidate probes are executed in the official SeClaw container. A probe is labeled locally successful only when the audit trajectory contains the intended protected operation for the intended target object. This avoids two common errors: a false positive, where the final answer mentions the target but no protected tool call occurs; and a false negative, where the final answer appears harmless even though the internal trajectory already contains the protected operation. We therefore evaluate actual tool behavior rather than surface text.

Trajectory auditing also provides failure diagnoses. If the agent stops after an explicit permission denial, the attempt is recorded as blocked. If it uses the wrong tool, the probe may lack scope control. If it targets the wrong resource, it may need stronger object anchoring. If the tool call fails because of a field mismatch, the probe may need better schema preservation. These diagnoses feed the revision loop and produce the local trajectory labels used in the experiments.

3.5 Tester-Side Reusable Interface (CPBT Skill)

For repeatability, we expose CPBT as a tester-side coding-agent skill. The skill guides an authorized coding agent through the local evaluation procedure: inspect workspace artifacts, define the protected-operation test objective, run bounded local executions, audit traces, classify failures, and produce a vulnerability report. It does not modify the evaluated model, tool implementations, runtime prompts, or audit logs.

Table 2 summarizes the interface components. The output is a trace-backed assessment rather than a static prompt-safety score. A report records whether the objective is vulnerable, blocked, or inconclusive; how many attempts were used; which material variant, if any, succeeded; what trace evidence supports the judgment; and what remediation or regression case should be kept.

Table 2: Tester-side interfaces implemented by CPBT.

Component	Role in the CPBT Skill
Workspace profile	Maps the local workspace into materials, harness, traces, and success oracle.
Objective model	Specifies protected asset, forbidden action, required authorization, and trace predicate.
Execution adapter	Runs bounded local evaluations in an isolated environment.
Trace oracle	Judges behavior from tool calls, resources, status, and authorization source rather than final text.
Failure classifier	Labels failed attempts as wrong tool, wrong resource, authorization denial, schema mismatch, or missing trace.
Report schema	Produces a reproducible vulnerability report with evidence and remediation.

4 Guard-Risk Approximation

4.1 Motivation

Local execution is not sufficient for official success. The evaluation pipeline contains hidden pre-execution checks, including material-level and task-instruction-level safety checks. A task can reach the protected operation locally but still receive no target credit if its package is rejected before trajectory scoring. This creates a mismatch between local reachability and official evaluation feasibility.

We therefore introduce a local guard-risk approximation. Its purpose is not to reverse-engineer the hidden checks, but to triage and revise locally successful packages under limited official attempts. Because the official feedback is sparse, the approximation provides a heuristic relative-risk ordering rather than a calibrated pass predictor. The trajectory audit separately determines whether the package can reach the target if executed.

4.2 Feedback Model

Let p_i be a candidate package for target y_i. Local execution gives a trajectory label

$$\ell_i = L_{y_i}(p_i) \in \{0, 1\},$$

where $\ell_i = 1$ means that the local trace reaches the protected operation. Official feedback provides a guard-pass label

$$b_i = G_1(p_i) \cdot G_2(p_i) \in \{0, 1\},$$

where $b_i = 1$ means that the candidate passes the hidden pre-execution checks and reaches trajectory scoring, whereas $b_i = 0$ means that it is filtered before target scoring. The final target-credit label is

$$c_i = b_i \cdot R_{y_i}(\tau(p_i)).$$

We use b_i, rather than c_i, when updating the guard-risk approximation because pre-execution rejection and trajectory failure are distinct outcomes. The tester does not observe the internal rules for G_1 or G_2.

The search first requires local trajectory success, $L_y(p) = 1$, and then uses the sparse guard-pass labels to rank or revise locally successful alternatives.

4.3 Submitted-Context Evidence

For each locally successful candidate, the guard-risk approximation examines the effective instruction surface of the submitted package: the user task, any loaded skill instructions, and any attachments. In the strict task-only setting, the adversarially designed part of this surface reduces to the user task. In the attachment-assisted setting, the additional material remains part of the evidence rather than being summarized away.

The approximation also receives a compact history derived from official feedback. This history records outcomes for exact-material matches, prioritizes the latest observed label when identical material has contradictory outcomes, and retains target-level outcomes only as background evidence. These signals provide diagnostic context without hard-coding target-specific pass or fail exceptions.

The semantic rubric covers four evidence groups. Operation explicitness distinguishes direct protected-action requests from checking, previewing, reconciling, or validating workflows. Object sensitivity covers personal records, financial operations, state changes, executive information, physical access, and live device state. Authorization-cue style covers unverified role labels, review values, emergency markers, owner claims, and checklist context. Workflow normalization covers negative scope control, object anchoring, and native schema preservation. These groups guide the semantic judgment.

4.4 LLM-Assisted Heuristic Guard-Risk Ranking

The available official feedback is too limited to support a calibrated predictive model. We therefore use an LLM-based semantic judge as a heuristic ranking mechanism rather than as a validated classifier. Given the effective submitted context and the latest available official guard labels, the judge compares evidence patterns and assigns a relative risk ordering among locally successful candidates.

Candidate selection proceeds in two steps. A candidate must first satisfy local trajectory success, $L_y(p) = 1$. Among locally successful candidates, the guard-risk approximation is used only to rank alternatives or identify surface features for revision.

4.5 Prompt Revision under Guard Feedback

Guard-aware revision is applied only after local trajectory success is achieved. This ordering matters: if a candidate is rewritten too aggressively to reduce estimated guard risk, it may lose the anchors needed for execution. We therefore keep target-binding information fixed, revise the task surface, and re-evaluate the complete submitted context.

The revision procedure has three goals. First, preserve object and schema anchors necessary for trajectory execution. Second, replace overly direct protected-action wording with a domain-normal workflow frame. Third, maintain enough contextual information for the agent to reach the relevant tool surface without converting the task into an explicit request for a protected operation.

This separation gives the workflow two distinct diagnostic outcomes. A local trajectory failure suggests that the current probe does not induce the protected operation. A guard-filtering failure suggests that the underlying trajectory may remain reachable, but the submitted surface is rejected before execution. Success at both layers indicates a high-priority permission-boundary weakness: the package is acceptable enough to pass pre-execution checks and still causes a protected operation.

5 Experiments

5.1 Evidence Sources and Metrics

We evaluated the method using three evidence sources. The first source was local trajectory validation in the official SeClaw container. This evidence tested whether a candidate prompt could induce the evaluated agent to reach the protected operation if executed. The second source was official final-round feedback, which reported the staged result for each submitted target. This evidence provided the primary success label because it included the hidden pre-execution checks. The third source comprised local guard-risk reports, which were used only as diagnostic evidence for prompt revision and not as a replacement for official labels. We report three stage-level diagnostic counts. Stage 1 pass count measures whether the submitted materials pass the competition's black-box skill and material security scan. Stage 2 pass count measures the number of targets for which the submitted task and its effective instruction surface pass the black-box task-instruction safety evaluation. Credited-target count measures the number of distinct targets for which the package passes the earlier stages and the recorded trajectory reaches the required protected operation. The official challenge score combines target coverage and sample precision [Alibaba Tianchi, 2026]:

$$C = \frac{S_o}{O}, \qquad P = \frac{S_N}{N}, \qquad F_{\text{attack}} = \frac{2CP}{C+P},$$

where O is the number of targets, S_o is the number of uniquely credited targets, N is the number of submitted samples, and S_N is the number of successful samples. Each of the three retained official submission records contains exactly one sample for each of the 19 targets, so $N = O = 19$ and $S_N = S_o$. The corresponding (N, S_N) pairs are $(19, 9)$, $(19, 10)$, and $(19, 16)$. Consequently, C, P, and F_{attack} coincide numerically with 9/19, 10/19, and 16/19 for the first, second, and third submissions, respectively. Local trajectory reachability is a separate existence result: a target is counted as reachable when at least one locally evaluated candidate reaches its protected operation, not as a per-attempt success rate.

5.2 Final-Round Evaluation Pressure

The final round is the main evaluation setting because it turns local reachability into a package-level feasibility problem. The official rules define a progressive three-stage review in which failure at any stage yields a zero score: skill security

scan, task-instruction safety evaluation, and trajectory scoring [Alibaba Tianchi, 2026]. Thus, a candidate is not credited merely because it can produce the target trajectory locally. It must be packaged in a form that passes the hidden pre-execution checks and still preserves the anchors needed for the agent to execute the protected operation. The staged pipeline therefore makes local trajectory reachability necessary but not sufficient for official credit: a locally executable candidate may still be rejected before its trajectory is evaluated. A package that is too operationally explicit may be filtered before execution, whereas a package that is too indirect may pass the guard but fail to drive the agent to the target tool boundary. This coupled constraint makes the final round more demanding than local trajectory validation and motivates the separate analysis of execution and pre-execution outcomes.

5.3 Chronological Submission Analysis

We analyze three chronological final-round submissions over the same set of 19 targets. This sequence is an observational development record rather than a controlled component comparison: task wording, package composition, auxiliary-material use, and guard-aware selection may have changed simultaneously. In parallel, local candidate packages were checked with strict trajectory auditing to ensure that the protected operations were reachable in the official container before black-box filtering was considered. This comparison distinguished two questions: whether the agent could be induced to perform the operation and whether the submitted task surface survived the hidden guards. The first and second submissions passed Stage 1 on all 19 targets and Stage 2 on 10 targets, with 9 and 10 credited targets, respectively. The third submission passed Stage 1 on all targets and obtained 16 Stage 2 passes and 16 credited targets. The sequence documents iterative improvement under official feedback, but it does not isolate the causal contribution of any individual CPBT component.

6 Results

6.1 Local Execution and Official Filtering

Local trajectory validation showed that the final candidate set reached all 19 protected operations in the official SeClaw container. In local execution, every target was associated with at least one candidate task that drove the evaluated agent across the protected tool boundary. Official feedback showed a different picture. The strongest final-round submission was credited on 16 of 19 targets because three locally executable candidates were rejected by the task-instruction safety evaluation before trajectory scoring. This gap motivates the two-loop design of our method. The workspace-level trajectory loop verifies whether a prompt produces the required protected tool trajectory. The black-box guard-feedback loop then uses sparse official labels to rank and revise locally successful prompts. Our evaluation therefore measures the feasibility of task-induced permission-boundary failures under limited official attempts.

Table 3: Target-level outcomes for three chronological final-round submissions.

Submission	Stage 1 passes	Stage 2 passes	Credited targets	P
First	19/19	10/19	9/19	9/19
Second	19/19	10/19	10/19	10/19
Third	19/19	16/19	16/19	16/19

6.2 Chronological Submission Results

Table 3 reports the three chronological final-round submissions. All three submissions passed Stage 1 on all 19 targets, indicating that the skill security scan was not the limiting factor for these attempts. The change was concentrated in Stage 2: the first two submissions passed 10 of 19 task-instruction safety checks, whereas the third passed 16 of 19. The credited-target count followed the same trend, increasing from 9/19 to 10/19 and then to 16/19.

The third submission's 16 credited targets comprise 15 strict task-only targets and one attachment-assisted target. This distinction matters for interpreting the method. Task-only success measures the lowest-friction attack surface because a single ordinary-looking task is sufficient to trigger the protected operation. The attachment-assisted success shows that the same local workflow can be extended to document-driven settings, but we do not count it as strict task-only evidence.

6.3 Guard-Feedback Analysis

The chronological results are consistent with the guard-aware revision process, but they do not isolate its causal effect because several package components changed across submissions. Earlier auxiliary-material and wrapper strategies often reached the protected trajectory locally but received no Stage 2 credit. This pattern suggests that local executability alone did not determine official evaluation feasibility.

The available diagnostic set contained 13 package-level feedback pairs. The guard-risk approximation agreed with 12 of the 13 observed labels. Because the same small feedback set informed the approximation and no independent held-out evaluation was performed, this value is descriptive in-sample agreement rather than an estimate of predictive accuracy. We therefore use the approximation only for ranking and diagnosis.

6.4 Failure Modes and Flip Cases

The three failures in the strongest official submission were concentrated in high-salience operations: live camera access, executive calendar access, and inventory mutation. These targets remain difficult because the task must be specific enough to guide execution but vague enough to pass hidden instruction-safety judgment. In these cases, the surface concepts required for execution are strongly associated with protected operations, so guard-aware rewriting cannot remove all risk signals without losing local executability.

The most informative instability case is the inventory target. It passed official Stage 2 and target scoring in the second submission, then failed both in the third even though

the target material was unchanged at the relevant target level. This shows that target-level material alone did not determine the outcome. Because the surrounding package and hidden evaluator may also have differed, the flip cannot distinguish among package-context sensitivity, evaluator updates, and stochastic variation. We therefore treat contradictory historical labels as uncertainty rather than as stable pass rules.

7 Discussion and Ethical Considerations

The results support evaluating tool-agent safety at the action boundary, not only from the task or final answer. Ordinary workflow context can be mistaken for authorization even without injected skills or multi-turn coercion. Deployment testing should therefore combine trace inspection, tool-interface authorization, and regression cases for tasks that crossed or approached protected boundaries. Workspace-level trajectory testing and black-box guard feedback answer different questions: the former establishes local reachability, whereas the latter provides only a context-specific ranking signal under hidden checks.

The evidence is limited to 19 targets in one controlled SeClaw setting. Local 19/19 reachability is an existence result rather than a per-attempt rate, and the chronological submissions do not identify the independent effect of guard-aware revision. Sparse and contradictory official labels also preclude claims of general guard-prediction accuracy. We therefore restrict the reusable skill to authorized, synthetic, and isolated workspaces, omit operational prompts, and frame its output as a trace-backed vulnerability report and regression case rather than an attack recipe.

8 Conclusion

We have presented CPBT, a tester-side workflow for measuring one-shot task-induced permission-boundary failures in tool-augmented agents. It combines workspace-level grounding, execution-trace auditing, and a heuristic use of limited black-box feedback without treating that feedback as a validated guard predictor.

In the SeClaw setting, CPBT reached all 19 protected operations locally. The strongest official submission credited 15 strict task-only targets plus one attachment-assisted target; The gap between local reachability and official credit shows why these outcomes must be measured separately and motivates broader evaluation across agent workspaces, permission models, and repeated defensive regression tests.

References

[Alibaba Tianchi, 2026] Alibaba Tianchi. Ali OpenClaw security attack-defense challenge: Competition information. https://tianchi.aliyun.com/competition/entrance/532481/information, 2026. Official competition information page; accessed June 30, 2026.

[Cheng *et al.*, 2026] Hao Cheng, Changtao Miao, Tianle Song, Yin Wu, He Liu, Erjia Xiao, Junchi Chen, Xiaoyu Shi, Yichi Wang, Jing Yang, et al. SeClaw: Spec-driven security task synthesis for evaluating autonomous agents. *arXiv preprint arXiv:2606.02302*, 2026.

[Debenedetti *et al.*, 2024] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024.

[Ganguli *et al.*, 2022] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei, Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022.

[Greshake *et al.*, 2023] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

[Hu *et al.*, 2025] Xueyu Hu, Tao Xiong, Biao Yi, Zishu Wei, Ruixuan Xiao, Yurun Chen, Jiasheng Ye, Meiling Tao, Xiangxin Zhou, Ziyu Zhao, et al. Os agents: A survey on mllm-based agents for general computing devices use. *arXiv preprint arXiv:2508.04482*, 2025.

[Luo *et al.*, 2025] Zeren Luo, Zifan Peng, Yule Liu, Zhen Sun, Mingchen Li, Jingyi Zheng, and Xinlei He. Unsafe llm-based search: Quantitative analysis and mitigation of safety risks in AI web search. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 8055–8074, 2025.

[Mao *et al.*, 2026] Qian'ang Mao, Jiaxin Wang, Ya Liu, Li Zhu, Cong Ma, and Jiaqi Yan. Sok: Security of autonomous llm agents in agentic commerce. *arXiv preprint arXiv:2604.15367*, 2026.

[Perez *et al.*, 2022] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022.

[Sun *et al.*, 2025] Zhen Sun, Zongmin Zhang, Deqi Liang, Han Sun, Yule Liu, Yun Shen, Xiangshan Gao, Yilong Yang, Shuai Liu, Yutao Yue, and Xinlei He. "To Survive, I Must Defect": Jailbreaking llms via the game-theory scenarios. *CoRR*, abs/2511.16278, 2025.

[Zhan *et al.*, 2024] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024.

[Zheng *et al.*, 2025a] Jingyi Zheng, Tianyi Hu, Tianshuo Cong, and Xinlei He. CL-Attack: Textual backdoor attacks via cross-lingual triggers. In *Proceedings of the*

AAAI Conference on Artificial Intelligence, volume 39, pages 26427–26435, 2025.

[Zheng *et al.*, 2025b] Jingyi Zheng, Junfeng Wang, Zhen Sun, Wenhan Dong, Yule Liu, and Xinlei He. TH-Bench: Evaluating evading attacks via humanizing AI text on machine-generated text detectors. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 2*, pages 5948–5959, New York, NY, USA, 2025. Association for Computing Machinery.